

Skill 13

1. **Task1:** To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

Code:

```
#include <stdio.h>
```

```
int add(int a, int b)
```

```
{  
    return a + b;  
}
```

```
int multiply(int a, int b)
```

```
{  
    return a * b;  
}
```

```
int main()
```

```
{  
    int a = 5, b = 3;  
  
    printf("Module integration test\n");  
    printf("add(%d,%d) = %d\n", a, b, add(a, b));  
    printf("multiply(%d,%d) = %d\n", a, b, multiply(a, b));  
    printf("End-to-end execution successful.\n");  
}
```

```
    return 0;
}
```

Outupt:

```
ashwithreddy_earla@AshwithReddy:~$ nano Skill13.c
ashwithreddy_earla@AshwithReddy:~$ gcc Skill13.c -o Skill13
ashwithreddy_earla@AshwithReddy:~$ ./Skill13
Module integration test
add(5,3) = 8
multiply(5,3) = 15
End-to-end execution successful.
ashwithreddy_earla@AshwithReddy:~$ |
```

Task2: To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

Code: `#include <stdio.h>`

`#include <errno.h>`

`#include <string.h>`

`int main()`

`{`

`FILE *fp = fopen("missing.txt", "r");`

`if (fp == NULL)`

`{`

`fprintf(stderr, "Runtime error: %s\n", strerror(errno));`

`fprintf(stderr, "Recovery: check the file path and try again.\n");`

```
        return 1;
    }

    fclose(fp);

    return 0;
}
```

OUTPUT:

```
End-to-end execution successful.
ashwithreddy_earla@AshwithReddy:~$ nano Skill13.1.c
ashwithreddy_earla@AshwithReddy:~$ gcc Skill13.1.c -o Skill13.1
ashwithreddy_earla@AshwithReddy:~$ ./Skill13.1
Runtime error: No such file or directory
Recovery: check the file path and try again.
ashwithreddy_earla@AshwithReddy:~$ |
```

Task3: To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

Code:

```
#include <stdio.h> #include <stdlib.h>

int main() { int *data = malloc(5 * sizeof(int));

if (data == NULL)
    return 1;

for (int i = 0; i < 5; i++)
    data[i] = i + 1;
```

```
printf("Allocated values: ");

for (int i = 0; i < 5; i++)
    printf("%d ", data[i]);

printf("\n");

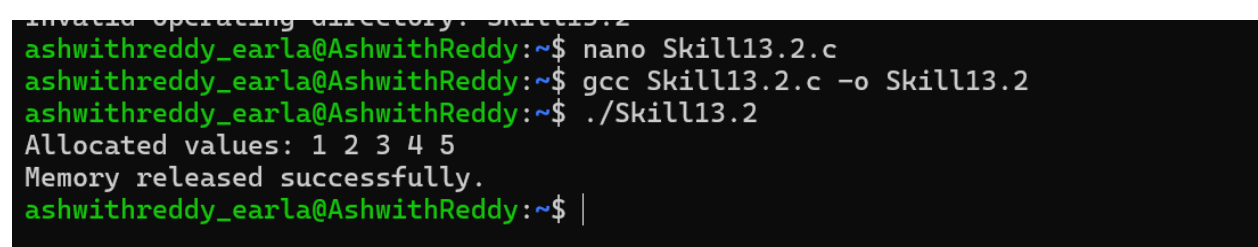
free(data);

printf("Memory released successfully.\n");

return 0;

}
```

OUTPUT :



```
invalid operating directory: Skill13.2
ashwithreddy_earla@AshwithReddy:~$ nano Skill13.2.c
ashwithreddy_earla@AshwithReddy:~$ gcc Skill13.2.c -o Skill13.2
ashwithreddy_earla@AshwithReddy:~$ ./Skill13.2
Allocated values: 1 2 3 4 5
Memory released successfully.
ashwithreddy_earla@AshwithReddy:~$ |
```

Task4: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

Code:

```
#!/bin/bash
```

```
gcc 13.4.c -o 13.4 || exit 1
```

```
output=$(./13.4)
```

```
expected="PASS"
```

```
if [ "$output" = "$expected" ]; then
```

```
    echo "Test 1: PASS"
```

```
else
```

```
    echo "Test 1: FAIL"
```

```
fi
```

```
echo "Test execution completed."
```

OUTPUT:

```
ashwithreddy_earla@AshwithReddy:~$ chmod +x test.sh
ashwithreddy_earla@AshwithReddy:~$ ./test.sh
Test 1: PASS
Test execution completed.
ashwithreddy_earla@AshwithReddy:~$ ls
Skill13.3.c  test.sh
ashwithreddy_earla@AshwithReddy:~$
```